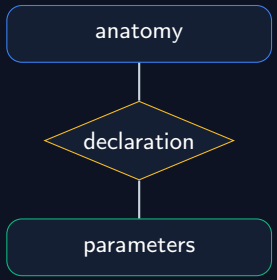
Functions in C/C++

Anatomy, Declaration and Parameters

Learn to structure code using subprograms and controlling how data is passed.

Week 7



Anatomy of a C/C++ function

BASIC STRUCTURE

A function is a subprogram that receives data, performs certain operations and, optionally, returns a result.

The general structure of a function includes:

- **Return type:** the data type of the result (or `void` if it returns nothing).
- **Function name:** the identifier used to call it.
- **Parameter list:** the variables that receive values upon calling (can be empty).
- **Function body:** the instructions between `{}`.

```
int sum(int a, int b) {
    int s = a + b;
    return s;
}

void greet() {
    cout << "Hello!";
}
```

Defining and declaring functions

HEADER AND IMPLEMENTATION

To be used, a function must be known to the compiler before being called. We have two distinct concepts:

Declaration (Prototype)

Represents the function header, followed by a semicolon `;`. It announces to the compiler the existence of the function, its return type, and its parameters, without giving the implementation.

```
int max(int x, int y);
```

Definition (Implementation)

Includes the function header and its body (between curly braces). A function cannot be defined multiple times, but it can be declared multiple times.

```
int max(int x, int y) {...}
```

Passing parameters by value

BY VALUE

When the function is called, the formal parameters (from the definition) receive a **copy** of the values of the actual parameters (from the call).

Warning:
Any modification of the parameters inside the function **will not affect** the original variables outside (in the calling program).

```
void double_val(int x) {
    x = x * 2;
    cout << "In function: " << x << endl
        ;
}

int main() {
    int a = 5;
    double_val(a); // Prints: 10
    cout << "In main: " << a; // Prints:
         5
    return 0;
}
```

Passing parameters by reference

BY REFERENCE

By using the & operator before the formal parameter's name, the function receives direct access to the memory area of the original variable.

Main advantage:

Modifications made to the formal parameter **are directly reflected** in the original variable passed at the call.

```
void swap(int &x, int &y) {
    int temp = x;
    x = y;
    y = temp;
}

int main() {
    int a = 3, b = 7;
    swap(a, b);
    // a becomes 7, b becomes 3
    return 0;
}
```

Summary exercise

TEST YOUR KNOWLEDGE

1. Modifying values

What will the following program print? Note the presence or absence of the & symbol.

```
void test(int a, int &b) {
    a++;
    b++;
}
int main() {
    int x = 1, y = 1;
    test(x, y);
    cout << x << " " << y;
}
```

Explanation

Correct answer: `1 2`

Why?

`x` is passed **by value**, so modifying parameter `a` does not affect variable `x`.

`y` is passed **by reference** (due to `&b`), so modifying parameter `b` is reflected in variable `y`, which becomes 2.